



SESSION 8

RESTFUL AND HTTP APIS

Learning Objectives

In this session, students will learn to:

- Describe an API and its purpose
- List the common types of APIs in Node.js and their functions
- Explain RESTful and HTTP APIs and their features
- Explain how to use RESTful API and HTTP API to manage data
- Identify the advantages and disadvantages of RESTful and HTTP APIs
- Distinguish between RESTful APIs and HTTP APIs

Application Programming Interfaces (APIs) translate a user's instructions into a format, which can be interpreted by the system to process the request and send back the response.

For example, an e-commerce application (say, App1) provides an option to pay through credit cards. To ensure the smooth transaction of the payment, App1 must talk to the credit card company's application (say, App2) that handles the authentication and processes payments made using credit cards. Each credit card company will have its payment processing application. For talking to App2, APP1 must use an API. The credit card company usually provides this API. This API will act as a middleman between App1 and App2 to

complete the transaction for the user.

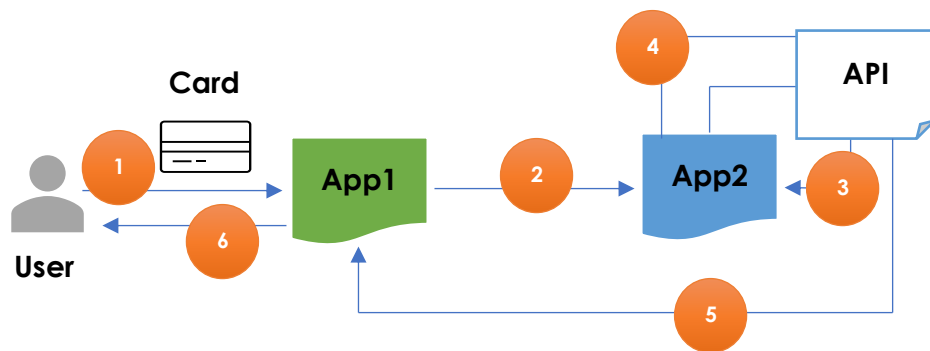


Figure 8.1: Payment Processing System

Figure 8.1 shows workflow of the payment processing system. In this case:

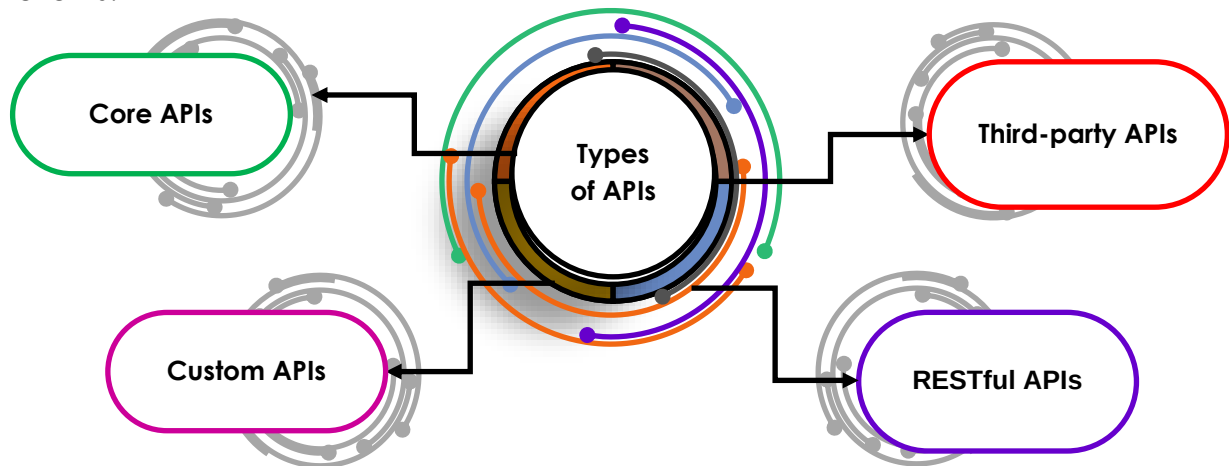
1. The user enters the card details on the payment Web page.
2. App1 sends the data to App2.
3. At App2, the details are received by the API, which, in turn, translates the data into a format that is understood by App2.
4. After the payment is processed, the API takes the status from App2 and translates it into a language that App1 understands.
5. App2 then, sends the payment status to App1.
6. App1 then, displays the status of the payment on the Web page for the user.

This session introduces common types of APIs in Node.js and their applications. It also lists features of Representational State Transferful (RESTful) API and HyperText Transfer Protocol (HTTP) API and their benefits and drawbacks. Finally, it explains applications of RESTful and HTTP APIs and differentiating features between RESTful and HTTP APIs.

8.1 Introduction to APIs

As already discussed, an API facilitates communication and interaction between various software applications through a set of rules and protocols. The main objective of an API is to organize the requests and their responses to enable the developers to extract data from other software applications.

Depending upon scope and usage, the APIs supported by Node.js are as follows:



- **Core APIs:** These APIs are provided by Node.js and are available as built-in modules, such as a filesystem module or HTTP module. The core APIs provide basic foundation blocks of an application. For example, consider that a user wants to build an e-commerce Website. Here, core APIs will provide basic functionalities that allow customers to browse the product catalog, handle shopping carts, manage product inventory, manage orders, and manage payments.
- **Third-Party APIs:** As the name suggests, these APIs are provided by other developers or companies that the users can integrate into their application to provide specialized features. These APIs are installed using Node Package Manager (`npm`). Some examples of third-party APIs include mongoose for connecting to MongoDB databases, Express.js to use Web frameworks, and Passport.js to handle authentication. In the e-commerce application, users can use third-party APIs to provide the product review feature. Customers can use this feature to provide product reviews and view reviews added by other customers.
- **Custom APIs:** These APIs are used to develop new functionality required by an application. The users can use the `module.exports` object to create functionality and make it available to other parts of the application. In an e-commerce application, custom APIs can be used to provide customers with personalized recommendations based on their browsing and purchasing history.
- **RESTful APIs:** As the name suggests, these APIs work on the principles of the Representational State Transfer (REST) architecture. They are type of HTTP APIs that use HTTP requests to access and manage data. These APIs define how data is shared between different parts of an application. In the e-commerce application, RESTful APIs can be used to ensure that data is transmitted in a standardized format such as JSON or CML.

The developer can choose a specific API based on the requirements of an application.

8.2 HTTP APIs

Node.js supports the HTTP API that uses HTTP protocol to facilitate communication and exchange of data between systems. These APIs use HTTP methods, such as GET, POST, PUT, and DELETE, for sending requests to the server. They use common formats, such as JSON and XML, to send back responses to the client.

Some of the features of HTTP APIs are as follows:

Client-server Model	They maintain clear distinction between client and server. In this model, the client is responsible for requesting actions or data and the server is responsible for processing and responding to the requests.
Layered Architecture	They allow multiple intermediary systems to participate in the communications between the client and server. This architecture offers flexibility by supporting distribution of processing responsibilities among various systems.
Caching	They allow intermediate systems, such as Web server and proxy servers, to cache responses for frequently requested data. This feature helps improve the performance of the application by reducing the load on the server.
Uniform Interface	They use a consistent interface based on the common HTTP methods and standard Uniform Resource Identifiers (URI) to work with data. This uniform interface makes it easy for developers to develop and maintain RESTful APIs applications.
Standard Media Types	They use standard media types, such as JSON and XML, for exchange of data between client and server. This feature makes HTTP API applications to be interoperable and compatible across diverse platforms.
Resource-centric Approach	They treat data as resources. These resources are identified by URIs and can be manipulated using HTTP methods. This feature makes the API structure easy to understand and implement.
Stateless Interactions	In these APIs, the server does not maintain any information related to client-specific state. This means that each request contains all the information required to understand and process the request, without any dependency on previous or next request.

Common methods used by HTTP APIs are as follows:

GET	POST	PUT	DELETE	PATCH
Used to send a request to retrieve data from the server	Used to send a request with the required information to the server to create a new resource	Used to send a request with the required information to the server to update an existing resource	Used to send a request to the server to remove a resource	Used to send a request to the server to partially update a resource

8.3 Advantages and Disadvantages of HTTP APIs

Table 8.1 lists some of the advantages and disadvantages of HTTP APIs.

Advantages		Disadvantages	
Simplicity	These APIs are easy to understand, design, implement, and maintain. This is because they use standard HTTP methods and common data formats.	Complexity	These APIs are not suitable for working with complex queries or data aggregations that require processing multiple requests simultaneously.
Scalability	These APIs are scalable owing to their stateless nature and resource-oriented architecture. This feature allows these APIs to adapt to changing requirements without significant architectural changes.	Performance Limitation	These APIs work on the request-response cycle and use HTTP methods that are less expressive. This feature puts a limit on the functionality and complexity of applications. Therefore, these APIs are not suitable for applications that require real-time streaming or high-frequency transactions.

Advantages		Disadvantages	
Efficiency	These APIs provide efficient performance because responses to frequent requests can be cached, thereby reducing the load on the server, and providing quick responses to such requests.	Security Concerns-	These APIs depend on HTTP protocol for authentication and data encryption. However, these services provided by HTTP are weak and prone to attacks and breaches.
Flexibility	These APIs can easily adapt to varying requirements. They also support multiple platforms and languages.	Limited Error Handling	These APIs use HTTP status codes to convey errors. These codes do not provide enough information that can be used for identifying errors and troubleshooting them. So, developers are required to implement additional mechanisms for debugging.
Portability	These APIs are platform-independent because they use standard HTTP protocol and common data formats.	Limited Standardization	These APIs do not have any official specifications or validations. This can lead to inconsistencies and ambiguities among different implementations.

Table 8.1: Advantages and Disadvantages of HTTP APIs

8.4 RESTful APIs

A RESTful API is a Web API that uses HTTP protocol to exchange data between applications. It is a subset of HTTP API that follows the principles and limitations of the REST architecture. It is used to design networked applications, especially Web services. Similar to HTTP APIs, RESTful APIs use HTTP methods, such as `GET`, `POST`, `PUT`, and `DELETE`, to perform operations on the resources.

RESTful APIs provide the same features, advantages, and disadvantages as HTTP APIs.

8.5 RESTful APIs vs. HTTP APIs

The RESTful and HTTP APIs have many common features, advantages, and disadvantages. However, both differ from each other in certain aspects, such as design principles and standards,

Table 8.2 lists the differences between these two APIs.

RESTful API		HTTP API
Architectural styles	RESTful APIs follow specific architectural limitations and design principles of REST, which include resource-based strategy, standardized HTTP request methods, stateless communication between client and server, and uniform interfaces.	HTTP APIs do not follow the REST principles of design. It is flexible to suit the requirements of the applications, define the endpoints, and use the HTTP methods without many restrictions.
URL patterns	RESTful APIs are resource-based, where each URL represents a resource or collection of resources. The URL includes a resource identifier (ID) to specify the resource that must be accessed or manipulated.	HTTP APIs are more flexible, where the endpoints are not strictly resource-based and include resource identifiers that can be structured depending on the design of the API.
Design standardization	All RESTful APIs have standardized design patterns and fixed architectural styles.	HTTP APIs do not have the same level of standardization and show considerable variation in design patterns and architectural styles.
Manageable Granularity	RESTful APIs enforce granularity in code making it easy for developers to develop and maintain applications.	HTTP APIs do not enforce granularity in code, making it difficult

RESTful API		HTTP API
		to develop complex applications.
Versioning	RESTful APIs enforce versioning.	HTTP APIs do not enforce versioning.
Hypermedia As The Engine Of Application State (HATEOAS)	In RESTful APIs, HATEOAS is a design principle. This principle emphasizes on the use of hypermedia links in the response to guide clients through the API and discover available resources and actions.	In HTTP APIs, HATEOAS is an optional element that is implemented in specific applications as required.

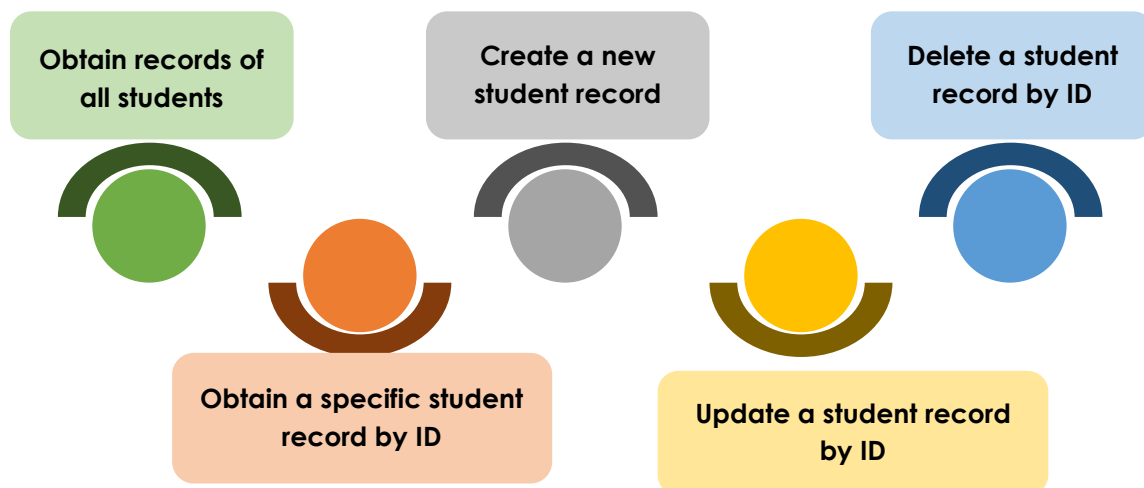
Table 8.2: Differences Between RESTful and HTTP APIs

8.6 Using HTTP APIs with Express.js

Express.js is an open-source Node.js Web application framework to create Websites and APIs. This framework handles different HTTP requests depending on the requirements.

Consider that a user wants to create a simple HTTP API to manage the data of students in a Node.js application using the Express.js framework.

This application will allow users to perform following tasks:



To run the application, the user wants to use cURL as the client to interact with the server. cURL is a tool that can be used to send requests to the server and receive responses from the server. To use cURL, users must have a command-line interface, in this case, Node.js, and an Internet connection.

To create this application, perform following steps:

1. Ensure that Node.js and Express.js are installed.
2. Open VS Code or a Notepad file.

3. To import the Express.js framework, create an instance of the Express.js application, and specify the port to be used for the Web application. Then, in the file, type the code given in Code Snippet 1.

Code Snippet 1:

```
const express = require('express');
const app = express();
const port = 3000;

app.use(express.json());
```

In Code Snippet 1, `require` function is used to import Express.js framework. Then, an instance of the Express application is created and stored in the variable `app`. Finally, the port to be used for this application is specified as `3000`.

4. To add some sample student data, in the same file, append the code given in Code Snippet 2.

Code Snippet 2:

```
// Sample data for students
let students = [
  { id: 1, name: 'David Miller', age: 12, grade: 'A' },
  { id: 2, name: 'Adam Smith', age: 13, grade: 'B' },
  { id: 3, name: 'John Williams', age: 12, grade: 'A' },
];
let nextStudentId = 4;
```

In Code Snippet 2, an array named `students` is created to store sample data of the students. Every student represents an object configured with `ID`, `name`, `age`, and `grade` properties. The variable `nextStudentId` initializes a value to assign unique IDs to new students. In this code, this variable is initialized to `4` as the first three students are already in the array.

5. To view details of all the students, in the same file, append the code given in Code Snippet 3.

Code Snippet 3:

```
// Get all students
app.get('/students', (req, res) => {
  res.json(students);
});
```

The code in Code Snippet 3 sends student details in JSON format when the endpoint of the request is `/students`.

6. To view the details of a student with a specific ID, in the same file, append the code given in Code Snippet 4.

Code Snippet 4:

```
// Get a specific student by ID
app.get('/students/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const student = students.find((s) => s.id === id);
  if (student) {
    res.json(student);
  } else {
    res.status(404).json({ message: 'Student not found'
  });
  }
});
```

Code Snippet 4 stores the ID received from the client in the `id` variable. It then searches for the student with the ID stored in the `id` variable and stores the student details found in the `student` variable. Finally, it uses an `if-else` block to return the student details if the student exists or return an error message if the student does not exist.

7. To add details of a new student, in the same file, append the code given in Code Snippet 5.

Code Snippet 5:

```
// Create a new student
app.post('/students', (req, res) => {
  const { name, age, grade } = req.body;
  const student = { id: nextStudentId++, name, age, grade
};
  students.push(student);
  res.status(201).json(student);
});
```

Code Snippet 5, first parses and stores the student details received from the client in an array. It then creates a student using the details in the array and the ID stored in the `nextStudentId` variable and then, increments this variable. It finally sends the new student details to the client in JSON format.

8. To update the details of a student with a specific ID, in the same file, append the code given in Code Snippet 6.

Code Snippet 6:

```
// Update a student by ID

app.put('/students/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const student = students.find((s) => s.id === id);

  if (!student)
  {
    res.status(404).json({ message: 'Student not found'
  });
  }
  else
  {
    const { name, age, grade } = req.body;
    student.name = name;
    student.age = age;
    student.grade = grade;
    res.json(student);
  }
});
```

Code Snippet 6 parses the ID received from the client and stores it in the `id` variable. It then uses the ID in the `id` variable to search for the student. It stores the details of the student found in the variable `student`. It then checks if the `student` variable exists. If it does not exist, an error message is sent back to the client. If the `student` variable exists, then the student details are replaced with the details received in the client request. These updated student details are then sent back to the client in JSON format.

9. To delete the details of a student with a specific ID, in the same file, append the code given in Code Snippet 7.

Code Snippet 7:

```
// Delete a student by ID
app.delete('/students/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = students.findIndex((s) => s.id === id);
  if (index !== -1) {
    students.splice(index, 1);
    res.json({ message: 'Student deleted' });
  } else {
    res.status(404).json({ message: 'Student not found'
  });
  }
});
```

Code Snippet 7 parses the ID received from the client and stores it in the `id` variable. It then, uses the ID in the `id` variable to search for the index of the student. It stores the index of the student found in the variable `index`. It then, checks if the `index` variable has a value that is not equal to `-1`. If the condition is met, the `splice` method is used to remove the student details at the specified index. The updated student details are then, sent back to the client in JSON format. If the condition is not met, an error message is sent back to the client.

10. To start the server and configure it to listen to port `3000`, in the same file, append the code given in Code Snippet 8.

Code Snippet 8:

```
// Start the server
app.listen(port, () => {
  console.log(`Server is running on port ${port}`);
});
```

11. Save the file as `studentHTTPapi.js` in the same folder where `Express.js` is installed.

Table 8.3 lists the standard HTTP methods and the endpoints defined in the code.

Operation	HTTP Method	Endpoint
To retrieve all student records	GET	<code>app.get('/students', (req, res)</code>
To retrieve a student record based on the ID	GET	<code>app.get('/students/:id', (req, res)</code>
To add a new student record in the database	POST	<code>app.post('/students', (req, res)</code>
To update the details of a student based on the ID	PUT	<code>app.put('/students/:id', (req, res)</code>
To delete a student record based on the ID	DELETE	<code>app.delete('/students/:id', (req, res)</code>

Table 8.3: HTTP Methods and Endpoints Defined for Application

12. To start the server, open a Command Prompt window.

13. At the Command Prompt, run the command as follows:

```
node studentHTTApi.js
```

The command starts the server. Once the server is up and running, a message is logged to the console indicating that the server is listening on the specified port, as shown in Figure 8.2.

```
C:\MyApps>node studentHTTApi.js
Server is running on port 3000
```

Figure 8.2: Starting the Server

The server is now ready to receive requests and send responses.

14. To use curl to interact with the Web server, open another Command Prompt window.
15. To obtain the records of all the students, at the command prompt, run the command as follows:

```
curl http://localhost:3000/students
```

This command displays information about all the students as shown in Figure 8.3.

```
C:\MyApps>curl http://localhost:3000/students
[{"id":1,"name":"David Miller","age":12,"grade":"A"},
{"id":2,"name":"Adam Smith","age":13,"grade":"B"},
{"id":3,"name":"John Williams","age":12,"grade":"A"}]
```

Figure 8.3: Information of all the Students

16. To obtain the records of a particular student by specifying the ID, at the Command Prompt, run the command as follows:

```
curl http://localhost:3000/students/2
```

The command specifies the ID as 2. The details of the student with ID as 2 are displayed as shown in Figure 8.4.

```
C:\MyApps>curl http://localhost:3000/students/2
{"id":2,"name":"Adam Smith","age":13,"grade":"B"}
```

Figure 8.4: Information of Student with Specified ID

17. To create a new student record, at the command prompt, run the command as:

```
curl -X POST -H "Content-Type: application/json" -d
{"name": "Michael Faraday", "age": 11, "grade":
"A"} http://localhost:3000/students
```

In this example, the `POST` request is used to create a new student record, with the information as follows:

- Name: Michael Faraday
- Age: 11
- Grade: A
- ID: 4

The command creates the new student record and displays the details as shown in Figure 8.5.

```
C:\MyApps>curl -X POST -H "Content-Type: application/j
son" -d '{"name": "Michael Faraday", "age": 11,
"grade": "A"}' http://localhost:3000/students
{"id":4,"name":"Michael Faraday","age":11,"grade":"A"}
```

Figure 8.5: Information of New Student

18. To check whether this student record has been created in the database, at the Command Prompt, run the command as follows:

```
curl http://localhost:3000/students
```

This command executes and displays the records of all the students as shown in Figure 8.6.

```
C:\MyApps>curl http://localhost:3000/students
[{"id":1,"name":"David Miller","age":12,"grade":"A"},
{"id":2,"name":"Adam Smith","age":13,"grade":"B"},{
"id":3,"name":"John Willams","age":12,"grade":"A"},{
"id":4,"name":"Michael Faraday","age":11,"grade":"A"
}]
```

Figure 8.6: New Student Added in the Database

19. To update a student record with a specific ID, at the Command Prompt, run the command as follows:

```
curl -X PUT -H "Content-Type: application/json" -d
"{\"name\": \"David Willams\", \"age\": 11, \"grade\":
\"A\"}" http://localhost:3000/students/3
```

In this example, the `PUT` request is used to update the information of the student with `ID` as 3. In this request, the `name` of the student is changed from John to David, and age is changed from 12 to 11. The command is executed, and the updated student information is displayed as shown in Figure 8.7.

```
C:\MyApps>curl -X PUT -H "Content-Type: application/
json" -d "{\"name\": \"David Willams\", \"age\": 11,
\"grade\": \"A\"}" http://localhost:3000/students/3
{"id":3,"name":"David Willams","age":11,"grade":"A"}
```

Figure 8.7: Updating Student Information

20. To check whether this student record has been updated in the database, at the Command Prompt, run the command as follows:

```
curl http://localhost:3000/students
```

This command executes and displays the records of all the students as shown in Figure 8.8.

```
C:\MyApps>curl http://localhost:3000/students
[{"id":1,"name":"David Miller","age":12,"grade":"A"},
{"id":2,"name":"Adam Smith","age":13,"grade":"B"},{
"id":3,"name":"David Willams","age":11,"grade":"A"},
{"id":4,"name":"Michael Faraday","age":11,"grade":"A
"}]
```

Figure 8.8: Student Information Updated in the Database

21. To delete a student record with a specific ID, at the Command Prompt, run the command as follows:

```
curl -X DELETE http://localhost:3000/students/4
```

In this example, the `DELETE` request is used to delete student record where the `ID` is 4. The command deletes the record and displays a message as shown in Figure 8.9.

```
C:\MyApps>curl -X DELETE http://localhost:3000/students/4
{"message":"Student deleted"}
```

Figure 8.9: Deleting Student Record

22. To check whether the student record has been deleted from the database, at the command prompt, run the command as follows:

```
curl http://localhost:3000/students
```

This command executes and displays the records of all the students as shown in Figure 8.10.

```
C:\MyApps>curl http://localhost:3000/students
[{"id":1,"name":"David Miller","age":12,"grade":"A"},
{"id":2,"name":"Adam Smith","age":13,"grade":"B"},{
"id":3,"name":"David Willams","age":11,"grade":"A"}]
```

Figure 8.10: Student Record Deleted from the Database

8.7 Using RESTful APIs with Express.js

Now, consider that the user wants to update the same student application to validate the data entered to create a new student. This can be done using RESTful API. In this case, a schema must be created and the data passed by the client must be verified against the schema.

To do this, the user must perform these steps:

1. Ensure that Node.js and Express.js are installed.
2. To use schema, open a Command Prompt window.
3. Navigate to the same folder where Express.js is installed.
4. To install the schema package, at the command prompt, run the command as follows:

```
npm install json-schema express-jsonschema
```

The command installs the package as shown in Figure 8.11.

```
C:\MyApps>npm install json-schema express-jsonschema
added 1 package, and audited 70 packages in 2s

11 packages are looking for funding
  run 'npm fund' for details

found 0 vulnerabilities
```

Figure 8.11: Installing the Schema Package

5. To install the validator package, at the Command Prompt, run the command as follows:

```
npm install express-validator
```

The command installs the package as shown in Figure 8.12.

```
C:\MyApps>npm install express-validator
added 3 packages, and audited 73 packages in 1s
11 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
```

Figure 8.12: Installing the Validator Package

6. To create the schema, open VS Code or a Notepad file and type the code given in Code Snippet 8.

Code Snippet 8:

```
{
  "$schema": "https://json-schema.org/draft/2019-09/schema",
  "$id": "studentSchema",
  "title": "Students",
  "description": "Student records",
  "type": "object",
  "properties": {
    {
      "id": {
        {
          "type": "integer",
          "description": "Unique identifier for the student",
          "required": true
        },
        "name": {
          {
            "type": "string",
            "description": "Full name of the student",
            "required": true
          },
          "age": {
            {
              "type": "integer",
```

```

        "description": "Age of the student",
        "required": true
    },
    "grade": {
        "type": "string",
        "description": "Grade of the student",
        "required": true
    }
}

```

In this schema, the details related to the students are defined as given in Table 8.4.

Detail	Data Type	Required
id	integer	true
name	string	true
age	integer	true
grade	string	true

Table 8.4: Schema Definitions

7. Save the file as `studentSchema.json` in the same folder where `Express.js` is installed.
8. Make a copy of the file `studentHTTPapi.js` and name it as `studentRESTfulapi.js`.
9. Open the `studentRESTfulapi.js` file.
10. To import the validator and reference the `studentSchema.json` file, update the code as given in Code Snippet 9.

Code Snippet 9:

```

const express = require('express');
const app = express();
const port = 3000;
const { check, validationResult } = require('express-validator');
const studentSchema = require('./studentSchema.json');
app.use(express.json());

```

The additional code in Code Snippet 9, specifies that the `express-validator` must extract the validation errors from the request and store it in the `validationResult` object. Additionally, the `studentSchema.json` file is imported and stored in the `studentSchema` object.

11. To update the code for creating a new student so that the data can be validated, update the code as given in Code Snippet 10.

Code Snippet 10:

```
/ Create a new student
app.post('/students', [
  // Using check for validation
  check('name').isString(),
  check('age').isInt({ min: 9, max: 13}),
  check('grade').isString(),
], (req, res) => {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    //Handling validation errors
    return res.status(400).json({ message: 'Errors in
entered data' });
  } else {
    const { name, age, grade } = req.body;
    const student = { id: nextStudentId++, name, age,
grade };
    students.push(student);
    res.status(201).json(student);
  }
});
```

In Code Snippet 10, the `check` function is used to validate if the data type of the details provided is the same as given in Table 8.4. The data type is checked using the `isString` and `isInt` methods. The code also checks if the age specified is between 9 and 13 by specifying the `min` property as 9 and the `max` property as 13. If all the data entered passes the validation, the new student is created and the new student details are displayed. Otherwise, an error message, `Errors in entered data`, is displayed.

12. Retaining the rest of the code as is, save the file.
13. To start the server, open a Command Prompt window.

14. At the command prompt, run the command as:

```
node studentRESTfulapi.js
```

The command starts the server. Once the server is up and running, a message is logged to the console indicating that the server is listening on the specified port, as shown in Figure 8.13.

```
C:\MyApps>node studentRESTfulapi.js  
Server is running on port 3000
```

Figure 8.13: Starting the Server

15. To create a new student record, at the Command Prompt, run the command as follows:

```
curl -X POST -H "Content-Type: application/json" -d  
"{\"name\": \"Michael Faraday\", \"age\": 11, \"grade\":  
\"A\"}" http://localhost:3000/students
```

The data provided in this command passes the validation rule. Therefore, the student is added to the database, as shown in Figure 8.14.

```
C:\MyApps>curl -X POST -H "Content-Type: application/json" -d  
"{\"name\": \"Michael Faraday\", \"age\": 11, \"grade\": \"A\"  
}" http://localhost:3000/students  
{\"id\":4,\"name\":\"Michael Faraday\",\"age\":11,\"grade\":\"A\"}
```

Figure 8.14: Adding a New Student

16. To check whether this student record has been created in the database, at the Command Prompt, run the command as follows:

```
curl http://localhost:3000/students
```

This command executes and displays the records of all the students as shown in Figure 8.15.

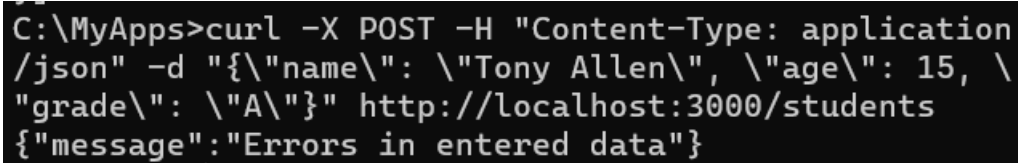
```
C:\MyApps>curl http://localhost:3000/students  
[{\"id\":1,\"name\":\"David Miller\",\"age\":12,\"grade\":\"A\"}  
, {\"id\":2,\"name\":\"Adam Smith\",\"age\":13,\"grade\":\"B\"}, {  
\"id\":3,\"name\":\"John Williams\",\"age\":12,\"grade\":\"A\"}, {  
\"id\":4,\"name\":\"Michael Faraday\",\"age\":11,\"grade\":\"A\"  
}]
```

Figure 8.15: New Student Added in the Database

17. To create another new student record, at the Command Prompt, run the command as follows:

```
curl -X POST -H "Content-Type: application/json" -d
{"name": "Tony Allen", "age": 15, "grade": "A"}
http://localhost:3000/students
```

In this command, the age is specified as 15, which is more than the maximum age, 13. Therefore, this data does not pass the validation and an error message is displayed, as shown in Figure 8.16.

A screenshot of a Windows Command Prompt window. The prompt is 'C:\MyApps>'. The command entered is 'curl -X POST -H "Content-Type: application/json" -d {"name": "Tony Allen", "age": 15, "grade": "A"} http://localhost:3000/students'. The output is '{"message": "Errors in entered data"}'.

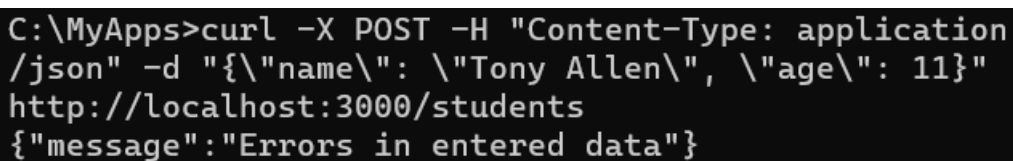
```
C:\MyApps>curl -X POST -H "Content-Type: application
/json" -d {"name": "Tony Allen", "age": 15, \
"grade": "A"} http://localhost:3000/students
{"message": "Errors in entered data"}
```

Figure 8.15: Error Message Displayed Due to Incorrect age

18. To create another new student record, at the Command Prompt, run the command as follows:

```
curl -X POST -H "Content-Type: application/json" -d
{"name": "Tony Allen", "age": 11}
http://localhost:3000/students
```

In this command, the grade is not specified. However, as per the schema, grade is a required detail. Therefore, this data does not pass the validation, and an error message is displayed, as shown in Figure 8.17.

A screenshot of a Windows Command Prompt window. The prompt is 'C:\MyApps>'. The command entered is 'curl -X POST -H "Content-Type: application/json" -d {"name": "Tony Allen", "age": 11} http://localhost:3000/students'. The output is '{"message": "Errors in entered data"}'.

```
C:\MyApps>curl -X POST -H "Content-Type: application
/json" -d {"name": "Tony Allen", "age": 11}
http://localhost:3000/students
{"message": "Errors in entered data"}
```

Figure 8.17: Error Message Displayed Due to Missing grade

8.8 Summary

- APIs translate a user's instructions into a format, which can be interpreted by the system to process the request and send back the response.
- An API facilitates communication and interaction between software applications through a set of rules and protocols.
- Depending upon the scope and usage, Node.js supports several APIs, such as core, third-party, custom, and RESTful APIs.
- HTTP API uses the HTTP protocol to facilitate communication and exchange of data between the systems.
- A RESTful API is an HTTP API that follows the principles and limitations of the REST architectural style to design networked applications.

Test Your Knowledge

1. You have created a simple Web application for a library. You want to find the details of a book using its name. Complete the code:

```
app.get('/books/:name', (req, res) => {  
  const bookName = req.params.name;  
  
  // Add missing code  
  
  if (book) {  
    res.json(book);  
  } else {  
    res.status(404).json({ message: 'Book not found' });  
  }  
});
```

- a) `const book = books.find((b) => b.name.toLowerCase())`
 - b) `const bookName = req.params.name;`
`const book = books.find((b) => b.name.toLowerCase() === bookName.toLowerCase());`
 - c) `books.find(bookName)`
 - d) `const bookName = req.params.name.find(Name)`
2. You are working on a Web application that must interact with a server to retrieve information. Which architecture will be useful for designing APIs to ensure simplicity, scalability, and statelessness?
- a) RESTful APIs
 - b) Custom APIs
 - c) Routing APIs
 - d) Sequential APIs
3. You are working on a Web application. While communicating with the server using HTTP APIs which method will you use to update the existing information on the server?
- a) GET
 - b) POST
 - c) DELETE
 - d) PUT

4. You are responsible for maintaining an online application for a hotel. In the hotel, there was a party and the manager entered wrong name of a guest, by mistake. You are required to update the guest list with the correct name. Which of the following command you will use to achieve this?

- a) Update Query
- b) `curl -X PUT -H "Content-Type: application/json" -d '{"key1": "value1", "key2": "value2"}' http://your-api-endpoint`
- c) `curl -X POST "Content-Type: application/json" -d '{"key1": "value1", "key2": "value2"}' http://your-api-endpoint`
- d) DELETE Query

5. Which of the following HTTP methods will you use to remove resources from the server?

- a) GET
- b) POST
- c) DELETE
- d) PUT

Answers to Test Your Knowledge

1	b
2	a
3	d
4	b
5	c

Try It Yourself

1. Create a simple RESTful API for managing employee details using Node.js and Express.

Table 8.5 lists the sample data for employees.

ID	Name	Age	Designation
1	Annie	27	Developer
2	Jack	30	Analyst
3	Paul	31	Developer
4	Harry	29	Sr. Analyst

Table 8.5: Employees Data

Install Express.js framework. Write a Node.js program to create the API for managing employee information.

- a) Create a schema for the employee information.
- b) Create an array using the data in Table 8.5.
- c) Define `GET` route to fetch all employee details.
- d) Define `GET` route for fetching the details of a specific employee by ID.
- e) Add details of a new employee.
- f) Validate the data entered.
- g) Update an employee by ID.
- h) Delete an employee by ID.
- i) Display the results on the console.
- j) Filter the employee data based on a specified `Designation` from the server when a `GET` request is made using a custom method.